



PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

October 26th, 2022

THE “in” OPERATOR

- Have used *in* operator in *for loop*
- Can be used for strings as logical operator
- It checks to see if one string is “in” another string
- The *in* expression is a logical expression that returns ***True*** or ***False*** and can be used in an if statement

THE “in” OPERATOR

- Example:

- fruit = 'banana'

- print('n' in fruit) → Output ?

- print('m' in fruit) → Output ?

- print('nan' in fruit) → Output ?

- if 'a' in fruit:

- print ('Found it!')

- else:

- print('Not found!') → Output ?

STRING COMPARISON

- Can use relational operators on strings
- Includes: == != < <= > >=
- Alphabetical order is considered for comparison results

STRING COMPARISON

- Example:
 - word = "apple"
if (word == "banana"):
 print("Alright, bananas!")
elif (word < "banana"):
 print("Your word, " + word + ", comes before banana")
elif (word > "banana"):
 print(("Your word, " + word + ", comes after banana"))
else:
 print("It's not bananas!")

STRING COMPARISON

- Output = ?
- Try for the following values:
 - word = "pineapple" → Output ?
 - word = "banana" → Output ?
 - word = "Banana" → Output ?
 - word = "Pineapple" → Output ?
 - word = "Apple" → Output ?
 - word = "bananas" → Output ?

STEP FUNCTION AND NEGATIVE INDEXING

- String can be sliced in steps as well
 - Example:
 - `print(str[0:6:2])` → Output = ?
- Negative indexing is allowed in Strings
 - Example:
 - `fruit = "banana"`
`print(fruit[-1])` → Output = ?
`print(fruit[-3])` → Output = ?

STRING METHODS

- A ***method*** works just like function with or without arguments and/or returning values
- But a string method is used with a different syntax
- Uses ***dot operator*** or ***dot notation*** to call the built-in string methods (similar to math library)
- In general, for library functions, we call them using the dot operator

STRING LIBRARY

- Python has a number of string methods which are in the string library
- These functions are already built into every string
 - Invoke them by appending the function to the string variable using the dot operator
- These functions do not modify the original string, instead they return a new string that has been altered

STRING LIBRARY

- String library functions include:
 - 'capitalize', 'center', 'count', 'decode', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'index', 'isalnum', 'isalpha', 'isdigit', 'islower', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill'
- Do not need to remember them all – use whenever needed

SEARCHING A STRING

- Searching for a specific letter in a string
- Can be done by using the ***in*** operator as discussed before
- But what if we want to know exactly where the letter is in the string, i.e. its index?
- We use the find() function to search for a substring within another string
- find() finds the first occurrence of the substring
- If the substring is not found, find() returns -1
- Remember that string position starts at zero

SEARCHING A STRING

- Example:
 - `fruit = 'banana'`
`pos = fruit.find('na')`
`print (pos)` → Output ?
`aa = fruit.find('z')`
`print (aa)` → Output ?

CHANGING CASE OF A STRING

- You can make a copy of a string in lower case or upper case
- Often when we are searching for a string using `find()` – we first convert the string to lower case so we can search a string regardless of case
- Example:
 - `greet = 'Hello Bob'`
`nnn = greet.upper()`
`print (nnn)` → Output ?
 - `www = greet.lower()`
`print (www)` → Output ?

CHANGING CASE OF A STRING

- You can make a copy of a string in lower case or upper case
- Often when we are searching for a string using `find()` – we first convert the string to lower case so we can search a string regardless of case
- Example:
 - `greet = 'Hello Bob'`
`nnn = greet.upper()`
`print (nnn)` → Output ?
 - `www = greet.lower()`
`print (www)` → Output ?
- Additional case functions include ***islower()*** and ***isupper()*** that returns ***True*** or ***False*** depending if all the letters in string are lower case and uppercase, respectively

CHANGING CASE OF A STRING

- You can make a copy of a string in lower case or upper case
- Often when we are searching for a string using `find()` – we first convert the string to lower case so we can search a string regardless of case
- Example:
 - `greet = 'Hello Bob'`
`nnn = greet.upper()`
`print (nnn)` → Output ?
 - `www = greet.lower()`
`print (www)` → Output ?
- Additional case functions include ***islower()*** and ***isupper()*** that returns ***True*** or ***False*** depending if all the letters in string are lower case and uppercase, respectively

EXERCISES

1. Write a function ***consonants_and_vowels*** that takes one argument, which is a string, and calculates the number of consonants and vowels in the function and prints them
2. Write a function ***slice_and_search*** that takes two arguments: **lst1** (which is a list of strings) and **str1** (which is a string). The function must use string slicing to search for the word from **lst1** in the given string, **str1**, and calculates the number of times it appears in the string. For each word in **lst1**, do the slice and search and prints the number of times that word appears in **str1**. For example:
 - **lst1** = ["Man", "Nova", "Ont", "NB", "Albert"]
 - **str1** = "ManitobanbmaNovaontScotialbertanabmaanOntarioOntNovAlbertaNewBrunswickNB"
 - The output will be:
 - The word: "Man" appears in the string: 2 number of times!
 - The word: "Nova" appears in the string: 2 number of times!
 - The word: "Ont" appears in the string: 3 number of times!
 - The word: "NB" appears in the string: 1 number of times!
 - The word: "Albert" appears in the string: 2 number of times!
3. Write a function that takes a hyphen separated string, **str1** as argument, and prints the hyphen separated string in alphabetically sorted form. For example, if **str1** = "YOU-LOVE-PAKISTAN", then the output should be = "OUIY-EOLV-AAIKNPST"

EXERCISE # 1 SOLUTION

1. Write a function ***consonants_and_vowels*** that takes one argument, which is a string, and calculates the number of consonants and vowels in the function and prints them
 - ```
def consonants_and_vowels(str1):
 consonants = "bcdfghjklmnpqrstvwxyz"
 vowels = "aeiou"
 consonant_count = 0
 vowel_count = 0

 for i in str1:
 if (i.lower() in consonants):
 consonant_count += 1
 elif (i.lower() in vowels):
 vowel_count += 1

 print(str(consonant_count) + " consonants and " + str(vowel_count) + " vowels")
```
  - ```
consonants_and_vowels("PFUN is so much fun!")
```

EXERCISE # 2 SOLUTION

2. Write a function ***slice_and_search*** that takes two arguments: **lst1** (which is a list of strings) and **str1** (which is a string). The function must use string slicing to search for the word from **lst1** in the given string, **str1**, and calculates the number of times it appears in the string. For each word in **lst1**, do the slice and search and prints the number of times that word appears in **str1**
- ```
def slice_and_search(lst1, str1):
 for i1 in lst1:
 i = i1.lower()
 count = 0
 wordlen = len(i)
 strlimit = len(str1) - wordlen

 for j in range(strlimit):
 slicelimit = j + wordlen
 sliceword1 = str1[j:slicelimit]
 sliceword = sliceword1.lower()

 if i == sliceword:
 count += 1

 print("The word: " + i1 + " appears in the string: " + str1 + " " + str(count) + " number of times!")
```
  - ```
l1 = ["Man", "Nova", "Ont", "NB", "Albert"]  
s1 = "ManitobanbmaNovaontScotialbertanabmaanOntarioOntNovAlbertaNewBrunswickNB"  
slice_and_search(l1, s1)
```

EXERCISE # 3 SOLUTION

```
def sort_string(str1):
    word = ''
    j = 0
    while (j < len(str1)):
        i = str1[j]
        substr = ''
        while(i != '-'):
            substr = substr + i
            j = j + 1
            if (j < len(str1)):
                i = str1[j]
            else:
                break

        sorted_word = sorted(substr)
        word = word + ''.join(sorted_word) + i

        if(j == len(str1)-1):
            break
        else:
            j = j + 1

    return word

w = sort_string("YOU-LOVE-PAKISTAN")
print(w)
```